Logic-based Artificial Intelligence, 4.0 VU, 192.204
Project Specification 2 - Answer Set Programming

The deadline for submitting your solution is on **Wednesday, June 17, 23:55 CEST**. You can submit multiple times, please submit early and do not wait until the deadline.

You can get up to **13 points** for this project. The submission will be checked automatically by running test cases. Detailed information on how to submit your project is below.

For questions concerning the assignment, please consult the TUWEL forum. Questions that reveal (part of) your solution should be discussed privately via emails to `lbai-2026s@kr.tuwien.ac.at`.

1 Task Assignment with ASP

The objective of this project is to construct a suitable *answer-set program* (ASP) for a scheduling problem, such that the answer-set of the program determines the solutions to this problem.

The evaluation of the constructed program shall be carried out with the `clingo` system[1]. You can download it from

`https://github.com/potassco/clingo/releases`.

Make sure to get a version $\geq$ `5.7.0`. Please use this tool as well for developing your solution.

The following links point to documents that might help you design the logic program:

- `http://ceur-ws.org/Vol-1145/tutorial1.pdf` (an introduction);
- `https://arxiv.org/pdf/1911.04326.pdf` (ASP-Core-2 standard);
- `https://github.com/potassco/guide/releases/` (Potassco User Guide).

You can achieve up to 13 points for submitting this project. Next, the general problem description for the project is specified.

1.1 General Problem Description

In the context of the 2026 Artemis III mission, a team of astronauts must conduct a series of scientific tasks at the Lunar South Pole. Each task is performed at different research stations and with different group of crew members.

We define the problem as follows: There are three tasks *site survey*, *drilling*, and *sample analysis* to be conducted by the astronauts in S number of research stations. The information of which tasks need to be conducted in which station is given. For each task, the astronauts are split into units of equal size U. An astronaut can conduct a task once and cannot be involved in two different tasks at the same station. The problem is to find an assignment such that no two astronauts perform a task together in a unit more than two times.

Furthermore, each astronaut has an expertise level from 1 to 3, belonging to an expertise class. To ensure a balanced distribution of skills, the schedule should have a minimum assignment of astronauts with the same expertise level to the same unit.

Example This instance has 9 astronauts. They perform each task in 3 out of 4 research stations in units of 3. Let the astronauts be $a, b, c, d, e, f, g, h, i$ with $\{a, b, c, d\}$, $\{e, f, g\}$, and $\{h, i\}$, denoting the expertise classes for levels 1,2, and 3, respectively. An optimal assignment would be:

[1] `https://potassco.org/clingo/`

	Station 1	Station 2	Station 3	Station 4
Site survey	b, f, i	d, e, h		a, c, g
Drilling		a, g, i	b, c, e	d, f, h
Sample analysis	c, g, h		a, d, f	b, e, i

Here we have, from those with same expertise level, $a\&d$, $a\&c$, and $b\&c$ being assigned to the same unit. We also have some astronaut pairs meeting twice, but not more.

2 Implementation

2.1 Definition of the Guess-and-Check-Program

Construct a program `schedule.lp` that produces schedules for given input instances that meet the described conditions above. The given input instance is assumed to be stored in input files containing the following data:

- the names of the participating astronauts;
- the names of the three tasks;
- the names of the research stations;
- the information which task should be conducted in which station;
- the unit size;
- the expertise information of the astronauts.

Furthermore, you must use the following predicates for the construction of the program `schedule.lp`, and for the input facts:

- `astronaut(A)`: `A` is a participating astronaut;
- `task(sitesurvey)`, `task(drilling)`, `task(sampleanalysis)`;
- `station(S)`: `S` is a research station;
- `unitsize(N)`: `N` is the number of astronauts within one unit;
- `conduct(S, T)`: at station `S` task `T` should be conducted ;
- `expertise(A, E)`: astronaut `A` has expertise level `E` $\in \{1, \ldots, 3\}$;
- `assigned(A, T, S)`: astronaut `A` performs task `T` at station `S`; **Use this predicate for the output of your program!**

Example Input Facts For the scenario above, the input facts would be defined as follows:

```
astronaut(a; b; c; d; e; f; g; h; i).
task(sitesurvey).  task(drilling).  task(sampleanalysis).
station(1..4).
unitsize(3).
conduct(1,sitesurvey).conduct(1,sampleanalysis).
conduct(2,sitesurvey).conduct(2,drilling).
conduct(3,drilling).conduct(3,sampleanalysis).
conduct(4,sitesurvey).conduct(4,drilling).conduct(4,sampleanalysis).
expertise(a,1).expertise(b,1).expertise(c,1).expertise(d,1).
expertise(e,2).expertise(f,2).expertise(g,2).
expertise(h,3).expertise(i,3).
```

The program `schedule.lp` should satisfy the following: Let P denote the union of `schedule.lp` with input facts as described above. Then, P computes answer-sets that represent optimal valid assignments. For the example above, one optimal answer-set would contain all facts of the form:

```
{assigned(b, sitesurvey, 1), assigned(f, sitesurvey, 1),
 assigned(c, sampleanalysis, 1), assigned(g, sampleanalysis, 1),...,
 assigned(a, drilling, 2)}.
```

Note, that the order of the predicates in the returned answer-set may be arbitrary. You can find a sample input file (describing the problem given before as an example) in the TUWEL course. Test your program with this input file to make sure that there are no typos in your predicate names.

2.2 Testing the Guess-and-Check Program

Test the functionality of your program `schedule.lp` by constructing *at least* five testcases. For this, save the data for each testcase n ($1 \leq n \leq 5$) in a file `test_`n`.lp` ($1 \leq n \leq 5$) containing all required facts as defined above (i.e., define facts about astronauts, tasks, stations, task per station information, expertise level of each astronaut, and the unit size). Note that it is beneficial to use both *positive tests* (containing facts about satisfiable instances) and *negative tests* (instances wit no possible answer-sets) for testing your program. Make sure to put comments on what the instance is testing.

With Clingo, you can run your implementation together with a testcase file, for example, `test_1.lp` containing your input by calling `clingo schedule.lp test_1.lp`.

2.3 Hints

Optimization For finding the minimum assignment, you can make use of `#minimize` statements provided in Clingo (not part of ASP-Core-2). They are written close to an aggregate syntax as:

$$\#minimize\{\ w@l, t_1, \ldots, t_n\ :\ b_1, \ldots, b_k, not\ b_{k+1}, \ldots, not\ b_m\ \}.$$

where w and l refer to the weight/cost and the priority level, respectively. For this project, we are not interested in multiple levels, thus l can be omitted.

For example, if one were trying to minimize the use of a specific resource, the statement would look like:

```
#minimize { 1,R,S : uses_resource(R, S) }.
```

This tells the solver to find an answer set that has the smallest possible number of unique `(R, S)` pairs that satisfy the `uses_resource` predicate. Further details can be found in the Potassco User Guide.

Useful information for debugging: In Clingo, you can limit the number of generated answer-sets to a specific number by using the command-line option `-n`, (e.g., `-n 20`). For generating all models, you can use `-n 0`. These might be useful for testing programs without optimization statements. For programs with optimization statements, in order to generate all optimal models, use `-opt-mode=optN`. You may want output only a specific set of predicates. In order to do so, put

```
#show myPredicate/arity.
```

in your test inputs. However, *do not* put these lines in `schedule.lp`! This is only for testing your own solution.

2.4 Submission

The strict deadline for the submission of this project is **Wednesday, June 17, 23:55 CEST**. Prior to the deadline, you can upload your solution several times. Submissions after the deadline will be ignored. There is no further submission possible after the official deadline for this project.

Submit your solution by uploading a ZIP file to the **TUWEL** platform ensuring that it contains the following files:

- `schedule.lp`
- A directory `instances` containing at least 5 different input instances named `test_`i with $i \in [1, \ldots, 5]$.

The encoding is tested against input instances as follows: `clingo schedule.lp <instance>` where `<instance>` is an instance file.

For the encoding, please remove all input facts since they are relative to a specific instance. We will test your encoding on different instances, which we will provide to Clingo in separate files.

The `#show` statement must *not* be included in the encodings. Make sure to properly rename your ASP encodings as described above, and that your ZIP file contains no subfolders.

It is important to document your answer-set programs properly using comments. You might not reach the maximum points if you do not comment your source code sufficiently or do not hand in the required 5 self-written testcases.

Make sure your code does not depend on input predicates that are not defined in the instruction or uses predicates not defined in `schedule.lp`; the program must be self-contained. Also make sure that your files are named correctly (no typos and the correct file-extension) and that your program does not yield duplicate answer-sets with respect to the set of `assigned`.

Furthermore, the program has to terminate for simple inputs within some seconds. (You can consider the previous example as an upper bound for the complexity of the testcases your program will be tested with.)

The number of points you receive for your submission depends on the number of inputs that work properly.

2.5 Questions

Please post questions of general interest to the TUWEL forum of the course. Solutions (or parts thereof) must not be revealed in the forum. Copying solutions is not allowed and can lead to the loss of all points for this project. The submissions will be checked for plagiarism!